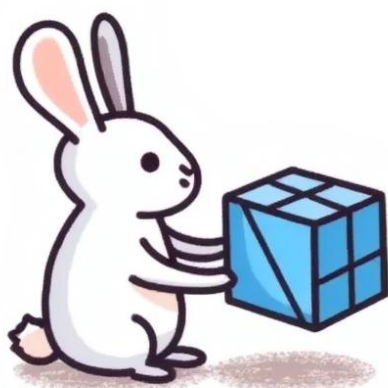


MLIR Part 2 - Memory in MLIR

Stephen Diehl

memory
management



MLIR's memory architecture represents a balance between high-level abstractions and low-level efficiency. At its core, MLIR provides three primary ways to represent data: **Tensors** for immutable, abstract operations; **MemRefs** for concrete memory buffers; and **LLVM-level constructs** for fine-grained control. This three-layer approach allows developers to work at their preferred level of abstraction while maintaining the ability to precisely control memory when needed.

While Tensors enable clean, mathematical expressions of computations, MemRefs provide a bridge to actual memory operations, and LLVM constructs offer direct control over memory layout and access patterns. This design is based on MLIR's philosophy of progressive lowering, where high-level concepts can be systematically transformed into more concrete representations as needed for optimization and execution.

In short, Tensors are pure mathematical objects, MemRefs are concrete memory buffers, and LLVM constructs are low-level memory operations, and vectors are references to multiple register values that we can manipulate simultaneously. Let's start with the highest level abstraction and work our way down.

Tensors

Tensors in MLIR are immutable, high-level data structures without explicit memory locations. They represent structured data in a computation-oriented manner designed for use in matrix-style computations often found in machine learning and linear algebra.

```

// 4-D static tensor with 10x10x10x10 elements of type f32
!TensorType1 = tensor<10 x 10 x 10 x 10 x f32>

// 4-D dynamic tensor with N x M x 10 x 10 elements of type f32
!TensorType2 = tensor<? x ? x 10 x 10 x f32>

// dummy example constants
%N = index.constant 2
%M = index.constant 3
%X = arith.constant 42.0 : f32

// allocate a tensor with the shape of the last 2 dimensions
%A = tensor.empty(%N, %M) : tensor<? x ? x 10 x 10 x f32>

// allocate a zero tensor with the shape of the last 2 dimensions
%B = tensor.empty() : tensor<10 x 10 x 10 x 10 x f32>

// create a tensor with the given values
%C = tensor.from_elements %X, %X, %X : tensor<3xf32>

// Common tensor operations
%D = tensor.extract %A[%N, %M, %N, %M] : !TensorType2
%E = tensor.insert %X into %A[%N, %M, %N, %M] : !TensorType2

```

Tensor operations facilitate mathematical transformations in a clean, [side-effect-free manner that makes analysis and optimization straightforward](#). For example, the identity matrix can be represented as:

$$A_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

```

func.func @identity(%m : index, %n : index) {
  %out = tensor.generate %m, %m {
    ^bb0(%i : index, %j : index):
      %ni = arith.index_cast %i : index to i32
      %nj = arith.index_cast %j : index to i32
      %elem = arith.cmpi eq, %ni, %nj : i32
      %elem_i32 = arith.extui %elem : i1 to i32
      tensor.yield %elem_i32 : i32
  } : tensor<?x?xi32>
  return
}

```

MemRefs

MemRefs are [MLIR's primary abstraction for representing memory buffers](#). They serve as a bridge between [high-level array concepts \(like NumPy arrays\) and low-level memory operations](#). A MemRef essentially

represents a pointer to a region of memory with additional metadata about its structure.

A MemRef consists of several core components, these are divided into *metadata* and *data*.

Allocated Pointer: Points to the data buffer (used for deallocation)

Aligned Pointer: Points to properly aligned data

Offset: Distance (in elements) between aligned pointer start and first accessible element

Shape: Array of integers describing dimensions

Strides: Array of integers describing memory layout

In MLIR, a MemRef type is declared using syntax like:

```
memref<1024xf32> // 1D array of 1024 float32 elements
memref<4x4xf64>  // 2D array of 4x4 float64 elements
```

Creating a Simple Array Addition Kernel

Let's create a simple example that adds two arrays using MLIR MemRefs:

```
module {
  func.func @array_add(%arg0: memref<1024xf32>, %arg1: memref<1024xf32>, %arg2:
memref<1024xf32>)
    attributes {llvm.emit_c_interface} {
      %c0 = arith.constant 0 : index
      %c1024 = arith.constant 1024 : index
      %c1 = arith.constant 1 : index

      scf.for %arg3 = %c0 to %c1024 step %c1 {
        %0 = memref.load %arg0[%arg3] : memref<1024xf32>
        %1 = memref.load %arg1[%arg3] : memref<1024xf32>
        %2 = arith.addf %0, %1 : f32
        memref.store %2, %arg2[%arg3] : memref<1024xf32>
      }

      return
    }
}
```

Then to compile the above MLIR code to an executable we can use the following commands:

```
mlir-opt array_add.mlir \
  -finalize-memref-to-llvm \
  -convert-scf-to-cf \
  -convert-cf-to-llvm \
  -convert-arith-to-llvm \
  -convert-func-to-llvm \
```

```

-o array_add_opt.mlir

mlir-translate array_add_opt.mlir \
  -mlir-to-llvmir \
  -o array_add_opt.ll

llc -filetype=obj --relocation-model=pic arary_add.ll -o arary_add.o
clang -shared -fPIC arary_add.o -o array_add.so

```

NumPy Array Memory Layout

Before we integrate with Python, it's important to understand how NumPy arrays are laid out in memory:

Data Buffer: Contiguous memory block holding the actual values

Shape: Tuple describing array dimensions

Strides: Tuple describing steps (in bytes) to move in each dimension

Dtype: Data type of elements

When you allocate a NumPy array it essentially creates a [PyObject wrapper](#) around a [raw memory buffer](#) with metadata about the shape, strides, and dtype of the array. We can move this memory back and forth between Python and the JIT compiled LLVM logic without copying the data.

```

import ctypes
import numpy as np

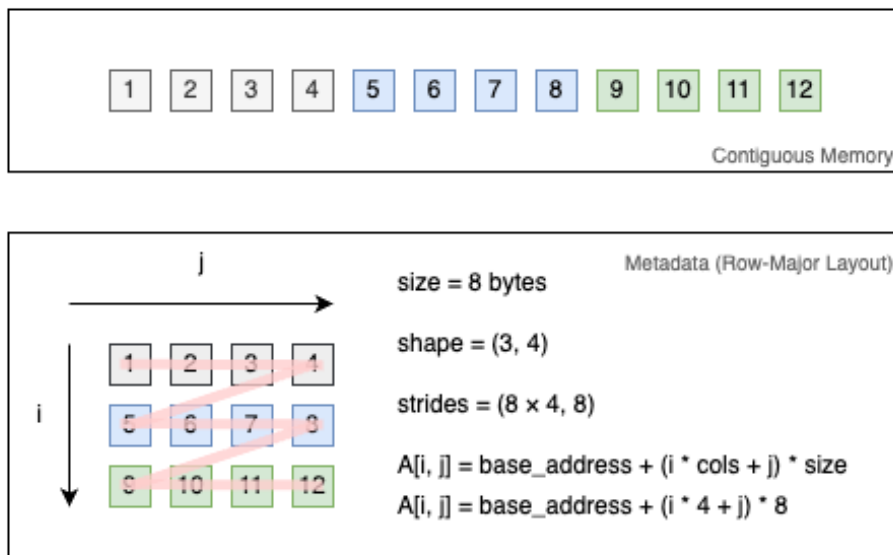
a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float32)
print(a.shape)           # (2, 3)
print(a.strides)          # (12, 4)
print(a.dtype)            # float32
print(a.itemsize)         # 4
print(a.ctypes.data)      # 0x7f926d6e2b20 (base pointer to the memory buffer)

# Get a ctypes pointer
print(a.ctypes.data_as(ctypes.POINTER(ctypes.c_float)))

```

By default numpy arrays are C-contiguous, which means that the memory is laid out in a row-major order. This is the most common memory layout and is the default for most NumPy operations. To calculate the memory address of the `[i, j]` element in a matrix we can use the following formula:

$$\text{address} = \text{base_address} + (i \times \text{strides}[0] + j \times \text{strides}[1])$$



For example, a row-major strided layout for `memref<3x4xf32>` is the same as both the following fully expanded memrefs (which are identical):

```
memref<3x4xf32, strided<[4, 1], offset: 0>>
memref<3x4xf32, affine_map<(i, j) -> (i, j)>>
```

Converting NumPy Arrays to MemRefs

To use NumPy arrays with our MLIR code, we need to create a bridge between NumPy's memory layout and MLIR's `MemRef` structure. We can do this by creating a Python class that matches MLIR's `MemRef` descriptor layout:

```
import numpy as np
from ctypes import c_void_p, c_longlong, Structure

class MemRefDescriptor(Structure):
    """Structure matching MLIR's MemRef descriptor"""

    _fields_ = [
        ("allocated", c_void_p), # Allocated pointer
        ("aligned", c_void_p), # Aligned pointer
        ("offset", c_longlong), # Offset in elements
        ("shape", c_longlong * 1), # Array shape (1D in this case)
        ("stride", c_longlong * 1), # Strides in elements
    ]

def numpy_to_memref(arr):
    """Convert a NumPy array to a MemRef descriptor"""
    if not arr.flags["C_CONTIGUOUS"]:
        arr = np.ascontiguousarray(arr)
```

```
desc = MemRefDescriptor()
desc.allocated = arr.ctypes.data_as(c_void_p)
desc.aligned = desc.allocated
desc.offset = 0
desc.shape[0] = arr.shape[0]
desc.stride[0] = 1

return desc
```

The `MemRefDescriptor` class mirrors the memory layout that MLIR expects:

`allocated`: Points to the start of the allocated memory

`aligned`: Points to aligned memory (often same as allocated)

`offset`: Number of elements to offset from the start

`shape`: Array dimensions (1D array in our example)

`stride`: Steps between elements in each dimension

The `numpy_to_memref` function handles the conversion by:

Ensuring the array is contiguous in memory

Creating a MemRef descriptor

Setting up the pointers to the NumPy array's data

Configuring the shape and stride information

This conversion allows us to [pass NumPy arrays to MLIR functions without copying the underlying data](#). Let's see how to use this in practice with our array addition example.

Integrating MLIR with Python

There are two main approaches to using MLIR code from Python: compiling to a shared library and loading it, or JIT-compiling the MLIR code at runtime. Let's explore both approaches using our array addition example.

Loading Pre-compiled MLIR Code

The first approach is to compile our MLIR code into a shared library that can be loaded from Python using `ctypes`. This requires compiling the MLIR code ahead of time:

```
# Compile MLIR to shared library
mlir-opt array_add.mlir \
  --convert-tensor-to-linalg \
  --convert-linalg-to-loops \
  --convert-scf-to-cf \
  --convert-cf-to-llvm \
  --convert-math-to-llvm \
  --convert-arith-to-llvm \
```

```
--convert-func-to-llvm \  
--convert-index-to-llvm \  
--finalize-memref-to-llvm \  
--reconcile-unrealized-casts \  
-o array_add_opt.mlir
```

```
mlir-translate --mlir-to-llvmir array_add_opt.mlir > array_add.ll  
llc -filetype=obj array_add.ll -o array_add.o  
clang -shared -fPIC array_add.o -o libarray_add.dylib
```

Then we can load and use the compiled library from Python:

```
import ctypes  
import numpy as np  
from np_memref import MemRefDescriptor, numpy_to_memref  
  
def main():  
    # Load the shared library  
    lib = ctypes.CDLL("./libarray_add.dylib")  
  
    # Get the C interface function  
    array_add = lib._mlir_ciface_array_add  
    array_add.argtypes = [  
        ctypes.POINTER(MemRefDescriptor)  
    ] * 3 # input1, input2, output  
    array_add.restype = None  
  
    # Create sample input arrays  
    size = 1024  
    a = np.ones(size, dtype=np.float32)  
    b = np.ones(size, dtype=np.float32) * 2  
    c = np.zeros(size, dtype=np.float32)  
  
    # Convert arrays to MemRef descriptors  
    a_desc = numpy_to_memref(a)  
    b_desc = numpy_to_memref(b)  
    c_desc = numpy_to_memref(c)  
  
    # Call the function  
    array_add(ctypes.byref(a_desc), ctypes.byref(b_desc), ctypes.byref(c_desc))  
  
    # Verify results  
    expected = a + b  
    np.testing.assert_array_almost_equal(c, expected)
```

```

print("Array addition successful!")
print(f"First few elements: {c[:5]}") # Should show [3.0, 3.0, 3.0, 3.0, 3.0]

if __name__ == "__main__":
    main()

```

This approach is simpler and more suitable for production use since the compilation is done [ahead of time](#). The compiled library can be distributed with your Python package and loaded at runtime.

JIT Compiling MLIR at Runtime

The second approach is to JIT-compile the MLIR code at runtime using **llvmlite** (Python bindings for LLVM). This gives more flexibility since the MLIR code can be generated or modified at runtime:

```

import os
import subprocess
import numpy as np
from ctypes import CFUNCTYPE, POINTER
from np_memref import MemRefDescriptor, numpy_to_memref

import llvmlite

# Disable typed pointers for LLVM 15+ compatibility
llvmlite.opaque_pointers_enabled = True

import llvmlite.binding as llvm # noqa: E402

# Initialize LLVM
llvm.initialize()
llvm.initialize_native_target()
llvm.initialize_native_asmprinter()

def compile_mlir_to_llvm(mlir_file_path):
    """Compile MLIR code to LLVM IR using mlir-opt and mlir-translate"""
    # Run mlir-opt to lower to LLVM dialect
    opt_cmd = [
        "mlir-opt",
        mlir_file_path,
        "--convert-tensor-to-linalg",
        "--convert-linalg-to-loops",
        "--convert-scf-to-cf",
        "--convert-cf-to-llvm",
        "--convert-math-to-llvm",

```



```

    "--convert-arith-to-llvm",
    "--convert-func-to-llvm",
    "--convert-index-to-llvm",
    "--finalize-memref-to-llvm",
    "--reconcile-unrealized-casts",
]

try:
    opt_result = subprocess.run(opt_cmd, capture_output=True, text=True,
check=True)
except subprocess.CalledProcessError as e:
    print("Error running mlir-opt:")
    print("STDOUT:", e.stdout)
    print("STDERR:", e.stderr)
    raise

# Run mlir-translate to convert to LLVM IR
translate_cmd = ["mlir-translate", "--mlir-to-llvmir"]
try:
    translate_result = subprocess.run(
        translate_cmd,
        input=opt_result.stdout,
        capture_output=True,
        text=True,
        check=True,
    )
except subprocess.CalledProcessError as e:
    print("Error running mlir-translate:")
    print("STDOUT:", e.stdout)
    print("STDERR:", e.stderr)
    raise

return translate_result.stdout

def create_execution_engine():
    """Create an ExecutionEngine suitable for JIT"""
    target = llvm.Target.from_default_triple()
    target_machine = target.create_target_machine()
    backing_mod = llvm.parse_assembly("")
    engine = llvm.create_mcjit_compiler(backing_mod, target_machine)
    return engine

```

```

def compile_and_load_mlir(mlir_file_path):
    """Compile MLIR code and load it into a JIT engine"""
    # Convert MLIR to LLVM IR
    llvm_ir = compile_mlir_to_llvm(mlir_file_path)

    # Create module from LLVM IR
    mod = llvm.parse_assembly(llvm_ir)
    mod.verify()

    # Create execution engine and add module
    engine = create_execution_engine()
    engine.add_module(mod)
    engine.finalize_object()

    return engine, mod


def main():
    # Get path to MLIR file
    current_dir = os.path.dirname(os.path.abspath(__file__))
    mlir_file = os.path.join(current_dir, "array_add.mlir")

    # Compile and load the MLIR code
    engine, mod = compile_and_load_mlir(mlir_file)

    # Get function pointer to the compiled function
    func_ptr = engine.get_function_address("_mlir_ciface_array_add")

    # Create the ctypes function wrapper
    array_add = CFUNCTYPE(
        None,
        POINTER(MemRefDescriptor),
        POINTER(MemRefDescriptor),
        POINTER(MemRefDescriptor),
    )(func_ptr)

    # Create test arrays
    size = 1024
    a = np.ones(size, dtype=np.float32)
    b = np.ones(size, dtype=np.float32) * 2
    c = np.zeros(size, dtype=np.float32)

```

```

# Convert arrays to MemRef descriptors
a_desc = numpy_to_memref(a)
b_desc = numpy_to_memref(b)
c_desc = numpy_to_memref(c)

# Call the JIT-compiled function
array_add(a_desc, b_desc, c_desc)

# Verify results
expected = a + b
np.testing.assert_array_almost_equal(c, expected)
print("Array addition successful!")
print(f"First few elements: {c[:5]}") # Should show [3.0, 3.0, 3.0, 3.0, 3.0]

if __name__ == "__main__":
    main()

```

The **JIT compilation** approach offers several advantages. [First, MLIR code can be generated or modified at runtime.](#) [Second, there is no need to manage separate compiled libraries.](#) [Finally, it allows for optimization for the specific CPU architecture at runtime.](#) However, this approach also has some drawbacks. [There is compilation overhead at startup, a more complex setup with dependencies on MLIR tools, and the need to handle compilation errors at runtime.](#)

Low-level LLVM Structures

In addition to the higher-level MLIR constructs, MLIR also provides a low-level LLVM dialect that allows for more direct control over memory layout and operations. Structs from LLVM dialect can be used to create custom types in MLIR. For example in C we would write:

```

struct Pair {
    int a;
    float b;
};

```

In MLIR we can define the same struct as follows:

```

!Pair = !llvm.struct<(i32, f32)>
%1 = llvm.mlir.zero : !Pair

```

This creates a new type alias `!Pair` which is a struct with two fields: an `i32` and a `f32`. [The `%1` variable is then an instance of this struct with all fields initialized to zero.](#)

LLVM arrays provide fixed-size sequential storage of elements. They can be created using the following syntax:

```

// Array of 10 integers
!IntArray = !llvm.array<10 x i32>

```

```
// Array of 4 pairs (using the struct defined above)
!PairArray = !llvm.array<4 x !Pair>

// Create and initialize an array
%arr = llvm.mlir.constant(dense<[1, 2, 3, 4]> : tensor<4xi32>) : !llvm.array<4 x i32>

// Access array elements
%idx = llvm.mlir.constant(2 : i32) : i32
%element = llvm.extractvalue %arr[2] : !llvm.array<4 x i32>

// Update array elements
%new_arr = llvm.insertvalue %element, %arr[0] : !llvm.array<4 x i32>
```

Unlike higher-level MLIR constructs like MemRefs, [LLVM arrays are more rigid and closely mirror the behavior of arrays in C](#). They have a fixed size known at compile time and don't carry any metadata about their structure.

Vectors

MLIR's vector dialect provides a low-level abstraction for **SIMD** (Single Instruction, Multiple Data) operations. While it's closely aligned with hardware vector capabilities, it's not strictly limited to what the target architecture supports, thanks to LLVM's vector operation lowering and optimization capabilities.

Vectors in MLIR can be:

Single-dimensional: representing traditional SIMD registers (usually operating over 2, 4, 8 elements)

Multi-dimensional: representing higher-order vector operations

```
// Single-dimensional vectors
vector<4xf32>      // 4-element vector of float32
vector<8xi32>      // 8-element vector of int32

// Multi-dimensional vectors
vector<2x4xf32>    // 2x4 matrix of float32 as vectors
vector<2x2x4xf32> // 3D vector arrangement
```

MLIR provides vector operations that map efficiently to hardware instructions:

```
// Vector arithmetic
%c = vector.add %a, %b : vector<4xf32>
%d = vector.mul %a, %b : vector<4xf32>

// Vector shuffles and extracts
%e = vector.extract %c[0] : vector<4xf32>
%f = vector.shuffle %c, %d [0, 1, 4, 5] : vector<4xf32>, vector<4xf32>

// Vector fold/reduction operations
```

```
%sum = vector.reduction <add>, %c : vector<4xf32> into f32
```

While MLIR's vector dialect provides operations that may not directly correspond to hardware capabilities, LLVM handles the lowering process by directly mapping to hardware vectors (e.g., AVX, NEON), splitting into smaller vectors, and scalarization when necessary.

For example, a `vector<8xf32>` operation might be mapped directly to AVX on x86, split into two NEON operations on ARM, or scalarized on architectures without vector support.

Bufferization

Bufferization in MLIR is the process of converting ops with tensor semantics to ops with memref semantics. The process of converting computations on the mathematical tensor construct to computations on physical memory buffers.

There are two low-level operations that are used to convert tensors to memrefs. They do what they say on the tin.

`bufferization.to_memref` - Convert a tensor to a memref

`bufferization.to_tensor` - Convert a memref to a tensor

```
%out1 = bufferization.to_memref %result : tensor<10xf32> to memref<10xf32>
%out2 = bufferization.to_tensor %result : memref<10xf32> to tensor<10xf32>
```

The `bufferization.materialize_in_destination` operation is used to convert a tensor to a memref in the destination buffer. This is equivalent to converting the tensor to a memref and then storing the result in the destination buffer but without the intermediate step. This is often the final step of a kernel that uses an externally allocated buffer.

```
bufferization.materialize_in_destination %0 in restrict writable %C :
(tensor<10xf32>, memref<10xf32>) -> ()
```

Now there are also passes that can do this conversion automatically. The `one-shot-bufferize` pass is a pass that can be used to convert all tensor operations to memref operations. Importantly this will not by default bufferize function boundaries (i.e. the input and output of a function are not bufferized), so we need to add the `bufferize-function-boundaries` flag to the pass.

```
mlir-opt -one-shot-bufferize="bufferize-function-boundaries"
```

A simple example demonstrates the transformation:

```
// Before bufferization
func.func @example(%arg0: tensor<8xf32>, %idx: index) -> tensor<8xf32> {
  %f = arith.constant 5.000000e+00 : f32
  %result = tensor.insert %f into %arg0[%idx] : tensor<8xf32>
  return %result : tensor<8xf32>
}

// After bufferization
func.func @example(%arg0: memref<8xf32>, %idx: index) -> memref<8xf32> {
  %f = arith.constant 5.000000e+00 : f32
```

```
memref.store %f, %arg0[%idx] : memref<8xf32>
return %arg0 : memref<8xf32>
}
```

In this case, the tensor insert operation was able to be bufferized in-place, reusing the input buffer for the output.

C-compatible wrappers

When we want to call a function from C, we need to emit a C-compatible wrapper. This is done using the `llvm.emit_c_interface` attribute. Normally when we bufferize a function we unroll all the memref fields into a single arguments. This is not desirable when we want to call the function from C where we would typically pass the memrefs as void pointers to structs.

```
func.func @add_vector_to_matrix(%A: memref<3xf32>) -> memref<3xf32> attributes
{llvm.emit_c_interface} {
  return %A : memref<3xf32>
}
```

This will emit a C-compatible wrapper for the function which is expanded into two functions.

```
module {
  llvm.func @fun(%arg0: !llvm.ptr, %arg1: !llvm.ptr, %arg2: i64, %arg3: i64,
%arg4: i64) -> !llvm.struct<(ptr, ptr, i64, array<1 x i64>, array<1 x i64>)>
attributes {llvm.emit_c_interface} {
    %0 = llvm.mlir.poison : !llvm.struct<(ptr, ptr, i64, array<1 x i64>, array<1 x
i64>)>
    %1 = llvm.insertvalue %arg0, %0[0] : !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    %2 = llvm.insertvalue %arg1, %1[1] : !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    %3 = llvm.insertvalue %arg2, %2[2] : !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    %4 = llvm.insertvalue %arg3, %3[3, 0] : !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    %5 = llvm.insertvalue %arg4, %4[4, 0] : !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    llvm.return %5 : !llvm.struct<(ptr, ptr, i64, array<1 x i64>, array<1 x i64>)>
  }
  llvm.func @_mlir_ciface_fun(%arg0: !llvm.ptr, %arg1: !llvm.ptr) attributes
{llvm.emit_c_interface} {
    %0 = llvm.load %arg1 : !llvm.ptr -> !llvm.struct<(ptr, ptr, i64, array<1 x
i64>, array<1 x i64>)>
    %1 = llvm.extractvalue %0[0] : !llvm.struct<(ptr, ptr, i64, array<1 x i64>,
array<1 x i64>)>
    %2 = llvm.extractvalue %0[1] : !llvm.struct<(ptr, ptr, i64, array<1 x i64>,
array<1 x i64>)>

```

```

array<1 x i64>)>
    %3 = llvm.extractvalue %0[2] : !llvm.struct<(ptr, ptr, i64, array<1 x i64>,
array<1 x i64>)>
    %4 = llvm.extractvalue %0[3, 0] : !llvm.struct<(ptr, ptr, i64, array<1 x i64>,
array<1 x i64>)>
    %5 = llvm.extractvalue %0[4, 0] : !llvm.struct<(ptr, ptr, i64, array<1 x i64>,
array<1 x i64>)>
    %6 = llvm.call @fun(%1, %2, %3, %4, %5) : (!llvm.ptr, !llvm.ptr, i64, i64,
i64) -> !llvm.struct<(ptr, ptr, i64, array<1 x i64>, array<1 x i64>)>
    llvm.store %6, %arg0 : !llvm.struct<(ptr, ptr, i64, array<1 x i64>, array<1 x
i64>)>, !llvm.ptr
    llvm.return
}
}

```

Instead now we call the `@_mlir_ciface_fun` function which is the C-compatible wrapper for the `@fun` function. Notice how it's given use two arguments, one for the input (`%arg0`) and one for the output (`%arg1`). Now we can pass our own memory buffers to the function and it will write the result back to our buffer with the code generation extracting the metadata from the memref.

External References

[MLIR Bufferization](#)

[One-Shot Function Bufferization of Tensor Programs](#)

[MLIR Open Meeting 2022-01-13: One-Shot Function Bufferization of Tensor Programs](#)

[MLIR Bufferization: From Tensors to MemRefs](#)

[How to init MemRef in MLIR](#)

[Tensor Layout](#)

[Tensor Operations](#)

[MLIR Dialect Conversion](#)

[NumPy Memory Layout](#)